# Updated Web Report

## FreeLancena Freelance Hiring Platform

Updated on May 18, 2026. This report reflects the current source code, project responsibilities, major features, backend routes, database models, email support, and Vercel deployment configuration.

## Project Overview

FreeLancena is a MERN-style freelance hiring platform for job seekers, employers, and admins. The frontend is built with React and Vite, and the backend is built with Express, MongoDB, Mongoose, JWT authentication, bcrypt password hashing, and Nodemailer email notifications.

The current application includes a modern home page, authentication pages, profile editing, job browsing, employer job creation, application submission, employer and job seeker dashboards, application decision tracking, admin statistics, and Vercel deployment configuration.

## Updated Main Features

- Home page now presents FreeLancena as a professional hiring platform with hero content, dashboard preview imagery, trust strip, feature cards, and links based on login state.
- Authentication supports register, login, demo users, JWT sessions, localStorage persistence, logout, and user context through AuthProvider.
- Users can maintain a profile with name, bio, skills, experience, and CV or portfolio link.
- Job seekers can browse jobs, search/filter listings, apply with contact details, CV filename or CV link, and a detailed message.
- Employers can create jobs, view applicants, review submitted messages/CV links, and mark applications as in process, accepted, or rejected.
- Email notification support sends application decision emails when an employer accepts or rejects an applicant. Console mode is supported for local/demo use.
- Admin users can view platform statistics and registered user data through protected admin routes.
- Deployment is configured for Vercel with the Express API under /api and the built frontend served as a static Vite app.

## Person 1: Authentication, Users, Profile

Owned frontend files: Login.jsx, Register.jsx, Profile.jsx, AuthContext.jsx, AuthContextObject.js, useAuth.js, Auth.css, and Profile.css.

Owned backend files: backend/routes/auth.js, backend/routes/users.js, backend/middleware/auth.js, and backend/models/User.js.

Login.jsx validates email and password, calls login() from useAuth(), shows loading/error states, and redirects successful users to the correct area. Admin users go to the dashboard, while normal users go to jobs.

Register.jsx collects name, email, password, confirm password, and account type. It validates the form, calls register() from AuthContext, stores the returned session, and redirects to jobs.

AuthContext.jsx is the main frontend session layer. It stores token and user in state

and localStorage, exposes login/register/logout/updateUser, supports demo accounts, and calculates isAuthenticated from the token.

Profile.jsx protects the page from unauthenticated users, loads profile data, and lets the user update name, bio, skills, experience, and CV or portfolio link.

User.js defines the MongoDB user model: name, email, hashed password, role, profile fields, lastLogin, loginCount, isActive, and timestamps.

auth.js handles POST /auth/register, POST /auth/login, GET /auth/me, user list helpers, and login statistics. It validates requests, hashes passwords with bcrypt, signs JWT tokens, and avoids returning passwords.

middleware/auth.js reads x-auth-token, verifies the JWT, stores decoded user data on req.user, and blocks invalid or missing sessions.

users.js provides profile read/update routes. The update route is protected and checks that the logged-in user can update only their own profile.

## Person 2: Jobs, Applications, Email

Owned frontend files: Jobs.jsx, CreateJob.jsx, Dashboard.jsx, Jobs.css, CreateJob.css, Dashboard.css, api.js, and config.js.

Owned backend files: backend/routes/jobs.js, backend/models/Job.js, and backend/utils/mailer.js.

Jobs.jsx loads all jobs, shows search and job type filters, calculates job/application summaries, and provides the job seeker application form. Applications collect name, email, phone, CV file name or CV URL, and a detailed message.

CreateJob.jsx protects job creation for employer accounts, validates title, company, location, description length, salary range, job type, experience level, and required skills, then calls jobApi.create().

Dashboard.jsx has two experiences. Employers see posted jobs, applicant cards, decision buttons, and charts. Job seekers see applied jobs and status tracking. Both sides share overview cards and visual charts.

api.js centralizes API calls, adds Content-Type and x-auth-token headers, wraps errors, and exposes authApi, jobApi, and userApi helper methods.

config.js uses /api in production and http://localhost:5001/api in development.

Job.js defines job fields plus embedded applications. Each application stores applicant userId, name, phone, email, cvName, cvUrl, message, status, and appliedAt.

jobs.js supports listing jobs, reading one job, creating employer-only jobs, applying as a job seeker, updating application status as the job owner, updating jobs, and deleting jobs.

mailer.js supports application decision emails. EMAIL_DELIVERY=console generates messages locally. SMTP mode uses SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_SECURE, and MAIL_FROM.

## Person 3: Admin, Layout, Deployment

Owned frontend files: Admin.jsx, Navbar.jsx, Footer.jsx, Home.jsx, App.jsx, main.jsx, Admin.css, Navbar.css, Footer.css, Home.css, global.css, App.css, and index.css.

Owned backend/deployment files: backend/routes/admins.js, backend/models/Admin.js, backend/server.js, backend/scripts/createAdmin.js, backend/scripts/viewUsers.js, backend/README.md, backend/package.json, backend/.env.example, api/index.js, vercel.json, and root package.json.

Home.jsx now acts as the first user-facing product screen. It includes a professional hero, images, role-aware calls to action, feature cards, and a hiring-process story section.

Footer.jsx uses the FreeLancena brand and provides platform links, employer links, contact email, Cairo location, and copyright information.

App.jsx defines the main routes: /, /login, /register, /jobs, /dashboard, /profile, /admin, and /create-job. Navbar and Footer wrap the routed pages inside AuthProvider.

Admin.jsx checks for a token, calls protected admin endpoints, shows user statistics, displays all registered users, and tracks admin login activity.

Admin.js defines admin accounts connected to User records, with superadmin/admin roles, permissions, active/inactive status, lastAdminLogin, adminLoginCount, and timestamps.

admins.js protects admin routes using JWT auth plus an isAdmin middleware. It supports dashboard stats, user listing, creating admins, listing admins, permission updates, removing admins, and admin login tracking.

server.js creates the Express app, configures CORS and JSON parsing, connects to MongoDB with a reusable connection promise, mounts /api/auth, /api/jobs, /api/users, and /api/admin, and exports the app for Vercel.

api/index.js exports the backend Express app so Vercel can run it as a serverless function. vercel.json routes /api/* to the API and all normal frontend routes to the Vite index.html fallback.

## Database Models

User model: stores the identity and account profile. Important fields are name, email, password, role, profile.bio, profile.skills, profile.experience, profile.resume, lastLogin, loginCount, isActive, createdAt, and updatedAt.

Job model: stores job posts and embedded applications. Important fields are title, description, company, location, salary, jobType, experienceLevel, skills, postedBy, applications, and timestamps.

Admin model: stores admin privileges for existing users. Important fields are userId, name, email, role, permissions, status, lastAdminLogin, adminLoginCount, createdAt, and updatedAt.

## Backend API Summary

Auth routes: POST /api/auth/register creates users, POST /api/auth/login verifies users, GET /api/auth/me returns current user, and stats/list helper endpoints support user reporting.

User routes: GET /api/users/:id returns a user without password, and PUT /api/users/:id updates profile data after JWT verification and ownership check.

Job routes: GET /api/jobs lists jobs, GET /api/jobs/:id returns a job, POST /api/jobs creates employer jobs, POST /api/jobs/:id/apply creates applications, PUT /api/jobs/:jobId/applications/:applicationId/status updates decisions, and PUT/DELETE /api/jobs/:id manage existing jobs.

Admin routes: GET /api/admin/dashboard returns counts, GET /api/admin/users returns registered users, POST /api/admin/create creates admins, GET /api/admin/list lists admins, PUT /api/admin/:adminId/permissions updates permissions, DELETE /api/admin/:adminId removes admins, and POST /api/admin/login-track updates admin login stats.

## Security and Session Handling

- Passwords are hashed with bcrypt before saving to MongoDB.
- JWT tokens include user id and role and expire after 7 days.
- Protected routes read x-auth-token and verify it before continuing.
- Profile updates check that the route id matches the authenticated user id.
- Employer-only job creation and job-owner-only application decisions are enforced in backend routes.
- Admin routes require both a valid token and an active Admin record.
- Production must provide a strong JWT_SECRET and MongoDB connection string through environment variables.

## Deployment and Environment

Root package.json defines build and vercel-build scripts that install frontend dependencies and build the Vite app.

vercel.json defines two builds: api/index.js with @vercel/node and frontend/package.json with @vercel/static-build. It routes /api/* to the backend and frontend routes to index.html.

backend/server.js uses MONGODB_URI or local mongodb://localhost:27017/jobportal. It only loads backend/.env locally, while Vercel uses configured environment variables.

Important environment variables include MONGODB_URI, JWT_SECRET, EMAIL_DELIVERY, SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_SECURE, and MAIL_FROM.

## Presentation Order

1. Person 3 starts with the project intro, home page, app routing, layout, backend server setup, MongoDB connection, and Vercel deployment.
2. Person 1 explains register/login, JWT authentication, sessions, User model, profile page, and profile updates.
3. Person 2 explains browsing jobs, creating jobs, applying, dashboards, accept/reject decisions, Job model, and email notifications.
4. Person 3 finishes with the admin panel, super admin permissions, environment variables, documentation, and final summary.

## Demo Accounts and Testing Notes

The frontend supports demo login credentials inside AuthContext.jsx: seeker@test.com / 123456, employer@test.com / 123456, and admin@test.com / 123456.

For real backend testing, MongoDB must be running or MONGODB_URI must point to a valid cloud database. For email testing, EMAIL_DELIVERY=console is easiest because it prints generated emails in the backend terminal.

Recommended smoke test: register a job seeker, register an employer, create a job as employer, apply as job seeker, return to employer dashboard, accept/reject the application, and confirm the status changes on the job seeker dashboard.

## Known Follow-up Check

One integration point to review during final testing: Profile.jsx calls userApi.getProfile(user.id), while api.js currently defines getProfile without a userId parameter and calls /users/profile. The backend route is GET /users/:id. The intended behavior is for profile loading to request /users/{userId}. Updating api.js to accept

the userId would align frontend and backend profile loading.

## Conclusion

The updated project now has a complete presentation story: Person 3 introduces the product, layout, routing, server, deployment, and admin system; Person 1 explains authentication, JWT sessions, users, and profile management; Person 2 explains the job marketplace, applications, dashboards, and email decisions. Together these features form a functional freelance hiring platform ready for demo and deployment.